

UADE

Exposición de experto

Programación III



Temas a desarrollar

1

Introducción a Greedy

2

Problema del Cambio

3

Problema de la Mochila

4

Código de Huffman

5

Algoritmos sobre Grafos

6

Dijkstra

7

PRIM

8

Kruskal



Greedy



Qué es la técnica de diseño de algoritmos Greedy

- Es una técnica de diseño de algoritmos que también se suele conocer como “Algoritmos Voraces”, por su traducción desde el inglés.
- Un algoritmo Greedy es aquel que va construyendo la solución a partir de decisiones parciales basadas en la información disponible en cada momento. No mira hacia adelante, es decir que no ve los efectos de las decisiones a futuro y nunca reconsidera una decisión ya tomada.
- En general se utilizan para resolver problemas de **optimización**.
- Suelen ser muy **eficientes** pero se debe validar su **correctitud** mediante una demostración teórica.

Algoritmos Greedy - Candidatos

- La estrategia de resolución de los problemas por Greedy se basa en identificar un conjunto de candidatos a formar parte de la solución.
- En cada paso se toma uno de los candidatos, el mas apropiado según un criterio definido para la solución del problema, y se evalúa si sirve o no, si sirve se agrega a la solución y si no se descarta.
- Para ello se debe poder saber en todo momento dado un candidato si esta pendiente de ser evaluado, si fue evaluado y agregado a la solución o si fue descartado.

Algoritmos Greedy - Estrategia

En un algoritmo Greedy se pueden identificar cuatro funciones:

- ***La función selección***: es la que selecciona el mejor candidato dentro de los pendientes.
- ***La función factibilidad***: evalúa si un candidato seleccionado es factible de formar parte de la solución.
- ***La función solución***: evalúa si un conjunto solución propuesto conforma la solución al problema.
- ***La función objetivo***: es la que define el resultado buscado de maximizar o *minimizar*.

Algoritmos Greedy - Esquema

ALGORITMO GREEDY

Entrada: C : conjunto de candidatos

Salida: S solución del problema

mientras $C \neq \emptyset$ Y NO *esSolucion*(S)

$x \leftarrow \text{Seleccionar}(C)$

$C \leftarrow C \setminus \{x\}$

 si *esFactible*($S \cup \{x\}$)

$S \leftarrow S \cup \{x\}$

 fin si

fin mientras

si *esSolucion*(S)

 devolver S

sino

 devolver No hay solución

fin si



Problema del Cambio



Problema del Cambio Greedy

El problema del cambio consiste en encontrar la forma de devolver un vuelto de valor v con monedas, utilizando la mínima cantidad de monedas. Las monedas son de denominaciones $d_1; d_2; \dots; d_n$, con una cantidad infinita cada denominación.

EJEMPLO

Monedas $\{\$25, \$10, \$100, \$50, \$5, \$1\}$

Si debemos dar un cambio \$ 1,77:

La mejor solución sería: $\{\$100, \$50, \$25, \$1, \$1\}$

Problema del Cambio *Greedy* - Estrategia

Conjunto candidatos	Las diferentes denominaciones de monedas
Función selección	Intentar seleccionar una moneda de mayor valor
Función factibilidad	Validar que la moneda seleccionada no supere el monto que debe pagarse
Función solución	Verificar que no se haya alcanzado el valor que queremos pagar
Función objetivo	Minimizar la cantidad de monedas a devolver para un valor dado

Problema del Cambio Greedy - Algoritmo

ALGORITMO CAMBIO

Entrada: v : entero

Salida: n : entero

entero $n \leftarrow 0$

entero $s \leftarrow 0$

entero $i \leftarrow 0$

Vector $monedas = [100, 50, 25, 10, 5, 1]$

mientras ($s < v$) Y ($i < longitud(monedas)$)

 si $s + monedas[i] < v$

$s \leftarrow s + monedas[i]$

$n \leftarrow n + 1$

 sino

$i \leftarrow i + 1$

 fin si

fin mientras

si $i < longitud(monedas)$

 devolver n

sino

 devolver No hay solución

fin si

Suponer el vector de monedas ordenado de mayor a menor.

Se comienza a utilizar la moneda de mayor valor, y se utiliza tantas veces mientras sea posible. Cuando esa moneda no se puede utilizar porque supera el valor a buscar se intenta con la siguiente.

En éste caso el algoritmo devuelve la cantidad de monedas, podría modificarse y devolver qué monedas, siendo la estrategia la misma

Problema del Cambio Greedy – Complejidad Temporal

ALGORITMO CAMBIO

Entrada: v : entero

Salida: n : entero

entero $n \leftarrow 0$

entero $s \leftarrow 0$

entero $i \leftarrow 0$

Vector $monedas = [100, 50, 25, 10, 5, 1]$

mientras $(s < v)$ Y $(i < longitud(monedas))$

 si $s + monedas[i] < v$

$s \leftarrow s + monedas[i]$

$n \leftarrow n + 1$

 sino

$i \leftarrow i + 1$

 fin si

fin mientras

si $i < longitud(monedas)$

 devolver n

sino

 devolver No hay solución

fin si

El análisis de complejidad esta dado por el ciclo, que en el peor caso puede iterar hasta v , que es el valor de la entrada, por lo tanto la complejidad temporal es:

$$O(n)$$

Se debe tener en cuenta que si el ordenamiento de las monedas se tuviera que realizar llevaría la siguiente complejidad temporal, siendo n en éste caso la cantidad de monedas

$$\Theta(n \log(n))$$

Problema del Cambio Greedy - Correctitud

- La solución al problema del cambio dada por Greedy para el conjunto de monedas dado {\$25, \$10, \$100, \$50, \$5, \$1} es correcto por las denominaciones particulares de monedas.
- Ahora si consideramos el caso {\$6, \$4, \$1} y debemos devolver por ejemplo \$8, el algoritmo daría como solución {\$6, \$1, \$1} cuando en realidad la solución es {\$4, \$4}, por lo tanto, deberemos analizar para éste caso si se puede resolver con otra técnica.

Nota: AHO, J.; HOPCROFT, John E. y ULLMAN, Jeffrey D.. Estructuras de Datos y Algoritmos. 1988. Wilmington : Addison Wesley Iberoamericana. 438 p. ISBN: 02016402449684443455.



Problema de la Mochila



Problema de la Mochila Greedy

El problema consiste en que se tienen n objetos y una mochila. Para $i = 1; 2; \dots, n$, el objeto i tiene un peso positivo P_i y un valor positivo V_i . La mochila puede llevar un peso que no sobrepase P . El objetivo es llenar la mochila de tal manera que se maximice el valor de los objetos transportados, respetando la limitación de capacidad impuesta.

Los objetos pueden ser fraccionados, si una fracción X_i ($0 \leq X_i \leq 1$) del objeto i es ubicada en la mochila contribuye en $X_i * P_i$ al peso total de la mochila y en $X_i * V_i$ al valor de la carga.

Problema de la Mochila Greedy - Ejemplo

Supongamos que tenemos 4 objetos y una mochila con capacidad $P = 17$. Los valores de los objetos son $(v_1, v_2, v_3, v_4) = (5, 2, 7, 4)$ y los pesos de los objetos son $(p_1, p_2, p_3, p_4) = (9, 5, 3, 6)$.

Algunas combinaciones para completar la mochila son:

x_1	x_2	x_3	x_4	$x_i \cdot p_i$	$x_i \cdot v_i$
1	1/2	1/2	1/2	17	11,50
1	1	1	0	17	14,00
1	0	1	1/2	17	14,00
8/9	0	1	1	17	15,44

La opción que maximiza respeta la relación v/p de mayor a menor:

Valor				Peso			
5	2	7	4	9	5	3	6
v/p				0,5556	0,4	2,3333	0,6667

Problema de la mochila *Greedy* - Estrategia

Conjunto candidatos	Los diferentes objetos aún disponibles a utilizar
Función selección	Seleccionar el objeto de mayor relación v/p
Función factibilidad	Validar que el objeto no supere el peso de la mochila, sino ingresar una fracción del mismo
Función solución	Verificar que la mochila se haya completado o se hayan utilizado todos los objetos
Función objetivo	Maximizar el beneficio obtenido en base a los objetos ingresados en la mochila

Problema de la Mochila Greedy - Algoritmo

ALGORITMO MOCHILA

Entrada: O : Vector<Objeto>, p : entero

Salida: R : Vector<real>

Ordenar(O) //según la razón valor/peso

para $i = 0$ hasta $n - 1$

$R[i] \leftarrow 0$

fin para

$suma \leftarrow 0$

$objeto \leftarrow 0$

mientras $suma < p$

$R[objeto] \leftarrow MIN(1, (p - suma)/O[objeto].peso)$

$suma \leftarrow suma + MIN(1, (p - suma)/O[objeto].peso) * O[objeto].valor$

$objeto \leftarrow objeto + 1$

fin mientras

devolver R

Problema de la Mochila Greedy – Complejidad Temporal

ALGORITMO MOCHILA

Entrada: O : Vector<Objeto>, p : entero

Salida: R : Vector<real>

$\text{Ordenar}(O)$ //según la razón valor/peso

para $i = 0$ hasta $n - 1$

$R[i] \leftarrow 0$

fin para

$\text{suma} \leftarrow 0$

$\text{objeto} \leftarrow 0$

mientras $\text{suma} < p$

$R[\text{objeto}] \leftarrow \text{MIN}(1, (p - \text{suma}) / O[\text{objeto}].\text{peso})$

$\text{suma} \leftarrow \text{suma} + \text{MIN}(1, (p - \text{suma}) / O[\text{objeto}].\text{peso}) * O[\text{objeto}].\text{peso}$

$\text{objeto} \leftarrow \text{objeto} + 1$

fin mientras

devolver R

$$\Theta(n \log(n))$$

$$\mathcal{O}(n)$$

La cantidad máxima que itera es utilizando todos los objetos

$$\mathcal{O}(n)$$

Complejidad del Algoritmo

$$\Theta(n \log(n))$$

Problema de la Mochila Greedy – Correctitud

La solución del problema de la mochila por Greedy para elementos que se pueden fraccionar es una solución correcta.

Se van a ver otras variantes del problema que se deben resolver con otras técnicas de diseño de algoritmo, para el caso de que los elementos no se puedan fraccionar.

Nota: BRASSARD, G.; BRATLEY, P. Fundamental of Algorithmics. 1996. Prentice Hall. xiii, 524 p. : ill. ISBN: 0133350681.



Código de Huffman



Código de Huffman

- Los códigos de Huffman se usan para comprimir información eficientemente y logran una reducción del 20% al 90% (dependiendo de la información original).
- La información es representada como una secuencia de caracteres, donde cada carácter tiene una frecuencia conocida. Por otra parte, se supone que cada carácter es representado en binario.
- El objetivo del problema es obtener la codificación de menor tamaño.

Código de Huffman – Longitud fija y variable

- Un código de longitud fija es un código en el que todas las palabras de código tienen el mismo largo. Ejemplo: codificamos $\{A, B, C, D, E, F\}$ con $A \rightarrow 000$, $B \rightarrow 001$, $C \rightarrow 010$, $D \rightarrow 011$, $E \rightarrow 100$ y $F \rightarrow 101$.
- En un código de longitud variable, las palabras de código tienen diferente longitud. Ejemplo: $A \rightarrow 0$, $B \rightarrow 101$, $C \rightarrow 100$, $D \rightarrow 111$, $E \rightarrow 1101$ y $F \rightarrow 1100$.
- En un código sin prefijo, ninguna palabra es prefijo de otra. Ambos códigos son considerados sin prefijo.
- Supongamos que en un mensaje con 10000 símbolos la frecuencia (en miles) es $A, 45$; $B, 13$; $C, 12$; $D, 16$; $E, 9$; y $F, 5$. El código de longitud fija requiere 30000 bits; La variable la longitud uno requiere 22400 bits.

Código de Huffman - Estrategia

Criterio de Solución por *Greedy*:

- **Conjunto de candidatos:** los símbolos a utilizar para la codificación de la palabra.
- **La función selección:** se comenzarán a trabajar con los símbolos con menor frecuencia.
- **La función factibilidad:** en este caso particular no habría una limitante a validar.
- **La función solución:** haber construido la codificación (árbol de Huffman).
- **La función objetivo:** minimizar el tamaño de la codificación.

Código de Huffman – Colas de prioridad

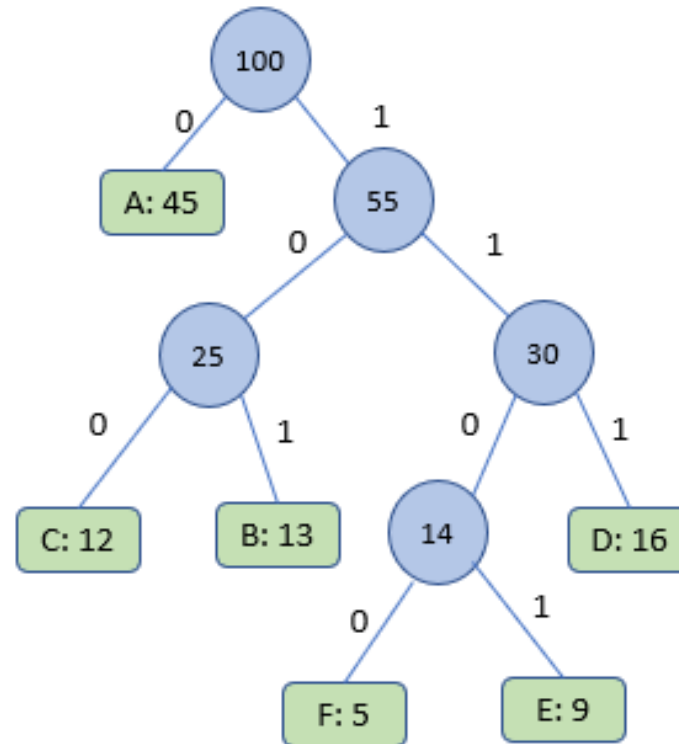
- Una cola de prioridad es una estructura de datos que contiene un conjunto S de elementos, cada uno de ellos asociado a una prioridad.
- Hay dos tipos de colas de prioridad: máxima y mínimo. Las respectivas implementaciones son simétricas.
- Las operaciones asociadas con las colas de menor prioridad (las cuales utilizaremos) son:
 - ✓ *Insertar* (S, x): inserta el elemento x en la cola S . La complejidad de esta operación es $\Theta(\log n)$.
 - ✓ *Extraer-Min*(S): devuelve el elemento x con la clave más baja y lo elimina de la cola S . La complejidad de esta operación es $\Theta(\log n)$.

Código de Huffman – Estrategia con árboles

- Para llevar a cabo la estrategia descripta anteriormente la idea es construir un árbol binario comenzando por las hojas, de los símbolos a codificar.
- El árbol está construido de abajo hacia arriba. Para cada símbolo, la ruta desde la raíz proporciona la codificación.
- Cada nodo interno del árbol está asociado con una clave de la cola de prioridad.

Código de Huffman – Ejemplo

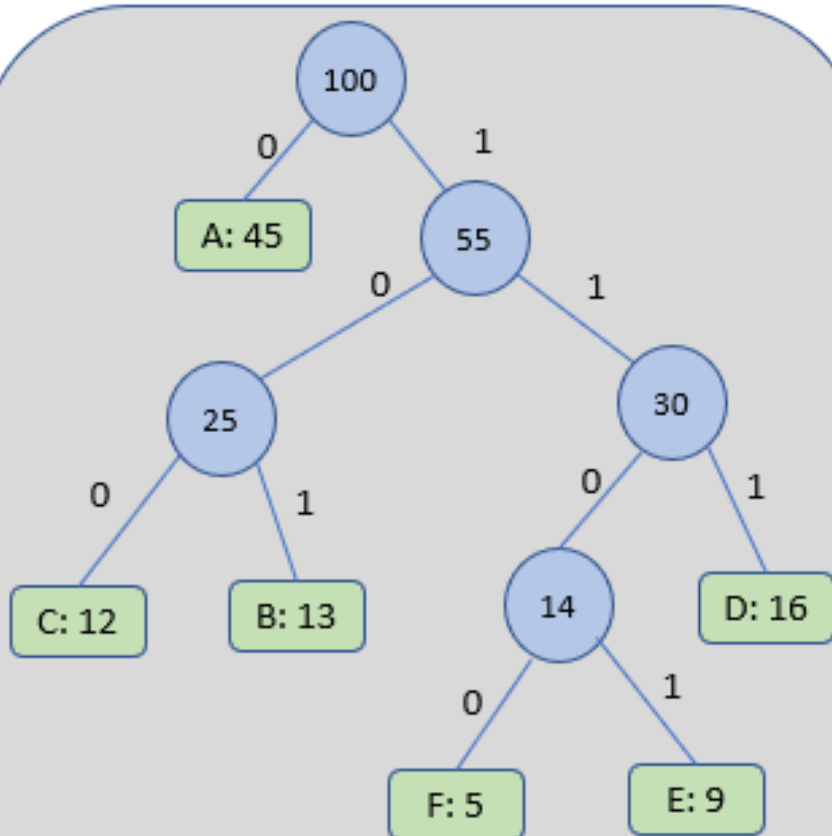
- Siguiendo el ejemplo anterior en donde en un mensaje con 10000 símbolos la frecuencia (en miles) es A, 45; B, 13; C, 12; D, 16; E, 9; y F, 5.



$A \rightarrow 0, B \rightarrow 101, C \rightarrow 100, D \rightarrow 111, E \rightarrow 1101, F \rightarrow 1100$

Código de Huffman – Ejemplo

¿Cómo se construye el árbol de Huffman? A continuación, se muestra:



<F:5, E:9, C:12, B:13, D:16, A:45>

<C:12, B:13, **X1:14**, D:16, A:45>

<**X1:14**, D:16, **X2:25**, A:45>

<**X2:25**, **X3:30**, A:45>

<A:45, **X4:55**>

<**X5:100**>

Código de Huffman - Algoritmo

ALGORITMO Huffman

Entrada: Conjunto C de símbolos

Salida: Z nodo raíz del árbol de codificación

```
n ← longitud (C)
Q ← InicializarColaPrioridad()
AgregarElementosACola(Q, C)
i ← 1
mientras ( i < n)
    x ← Extraer_Min(Q)
    y ← Extraer_Min(Q)
    z ← crearNodo()
    AgregarNodoIzq(z, x)
    AgregarNodoDer(z, y)
    z.frecuencia ← x.frecuencia + y.frecuencia
    Insertar(Q, z)
    i ← i + 1;
fin mientras
devolver z;
fin Algoritmo Huffman
```

Código de Huffman – Complejidad temporal

ALGORITMO Huffman

Entrada: Conjunto C de símbolos

Salida: Z nodo raíz del árbol de codificación

$n \leftarrow \text{longitud}(C)$

$Q \leftarrow \text{InicializarColaPrioridad}()$

$\text{AgregarElementosACola}(Q, C)$

$\Theta(n \log n)$

$i \leftarrow 1$

mientras ($i < n$)

$\Theta(n)$

$x \leftarrow \text{Extraer_Min}(Q)$

$\Theta(\log n)$

$y \leftarrow \text{Extraer_Min}(Q)$

$\Theta(\log n)$

$z \leftarrow \text{crearNodo}()$

$\text{AgregarNodoIzq}(z, x)$

$\text{AgregarNodoDer}(z, y)$

$z.\text{frecuencia} \leftarrow x.\text{frecuencia} + y.\text{frecuencia}$

$\text{Insertar}(Q, z)$

$\Theta(\log n)$

$i \leftarrow i + 1;$

fin mientras

devolver z;

fin Algoritmo Huffman

Complejidad del Algoritmo: $\Theta(n \log n) + \Theta(n) \Theta(\log n) \rightarrow \Theta(n \log n)$



Dijkstra



Greedy Grafos - Dijkstra

Problema de los caminos mínimos:

Dado un grafo y un nodo particular del mismo, buscar el costo del camino mas corto entre ese nodo y todos los demás, siempre que exista un camino.

El resultado es un grafo con los mismos nodos que el original pero solo hay aristas entre el nodo buscado y todos los demás y el peso de la arista es el costo del camino mínimo.

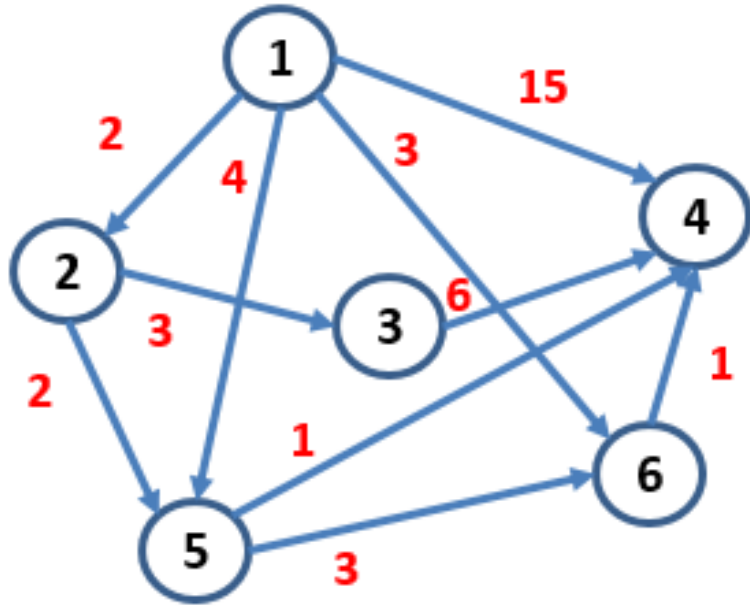
Para poder ejecutar el algoritmo es necesario contar con un grafo dirigido, con todas sus aristas con peso positivo.

Greedy Grafos – Dijkstra - Estrategia

- ***Candidatos:*** El conjunto de candidatos es el conjunto de vértices/nodos del grafo, sacando el nodo de origen.
- ***La función selección:*** En cada iteración, el método de selección será el encargado de elegir de todos los nodos pendientes de visitar, aquel que tenga el camino de menor costo desde el origen y lo marca como visitado, además de confirmar como definitivo el costo del camino mínimo desde el origen hasta dicho nodo.
- ***La función factibilidad:*** Calcular que si existe un camino directo el costo sea mayor a pasar por el nodo seleccionado, en caso de que exista camino pasando por ese nodo.
- ***La función solución:*** *Haber trabajado con todos los nodos candidatos*
- ***La función objetivo:*** Grafo con el valor de todos los caminos mínimos desde el nodo origen dado.

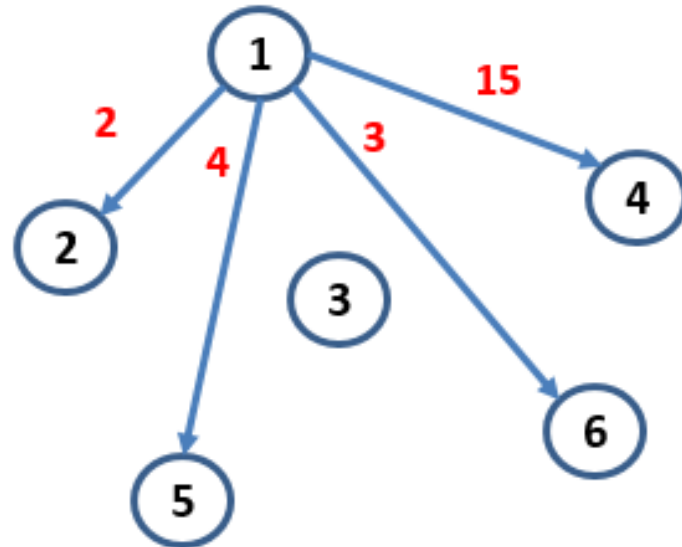
Greedy Grafos – Dijkstra - Ejemplo

Grafo Original



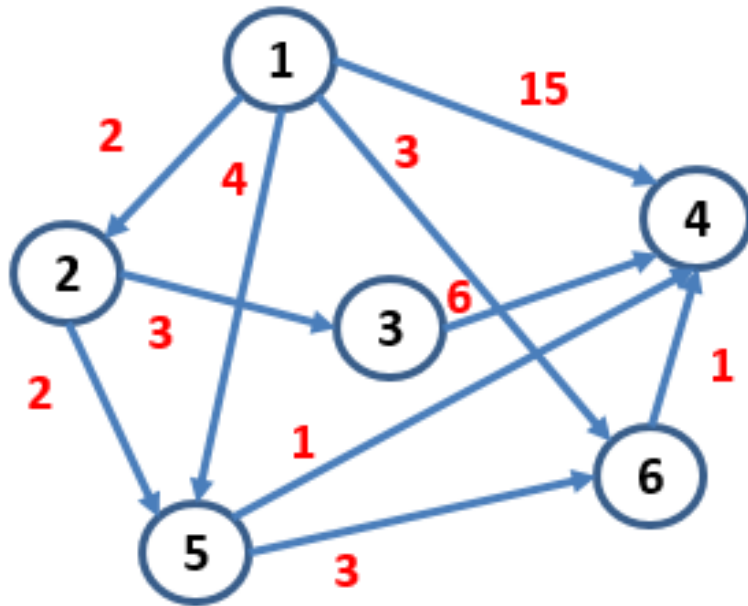
Calcular el camino mínimo desde el vértice "1" al resto

Grafo Resultado

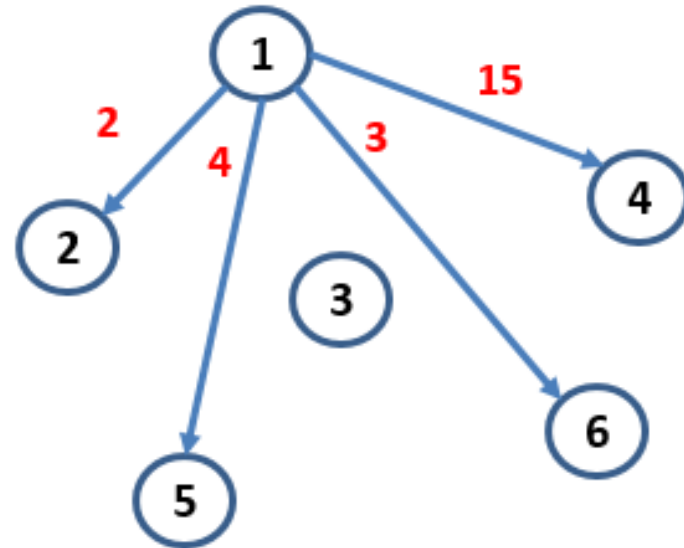


Greedy Grafos – Dijkstra - Ejemplo

Grafo Original



Grafo Resultado



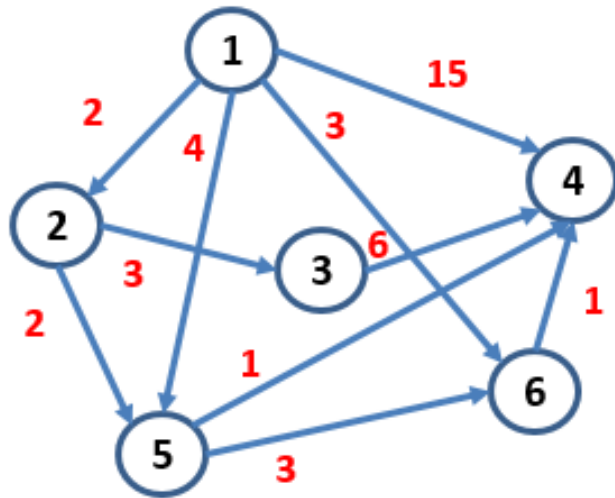
Candidatos desde "1" = {2, 3, 6, 5, 4}

Se selecciona el más cercano que es el nodo "2" por tener el peso menor "2"

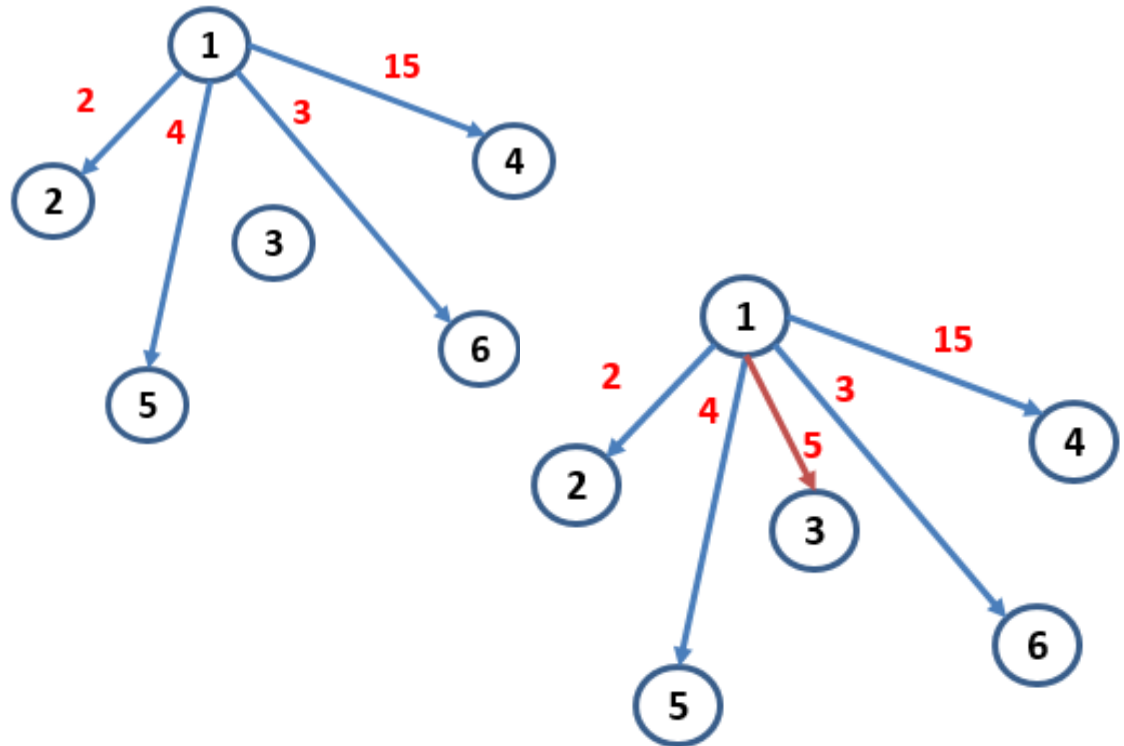
Recalculamos los caminos desde "1" al resto de los nodos pasando por "2" los caminos:

Greedy Grafos – Dijkstra - Ejemplo

Grafo Original



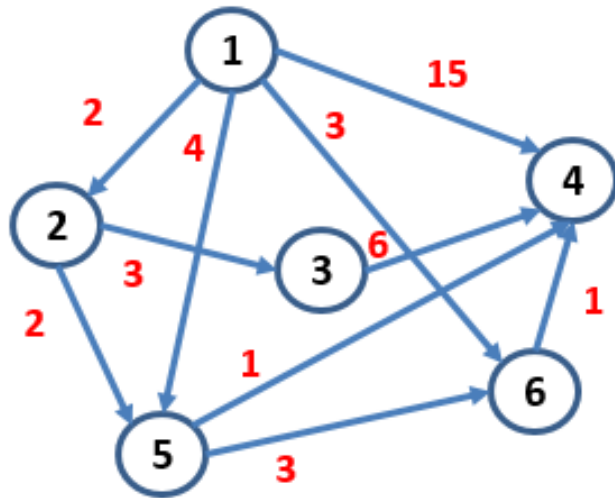
Grafo Resultado



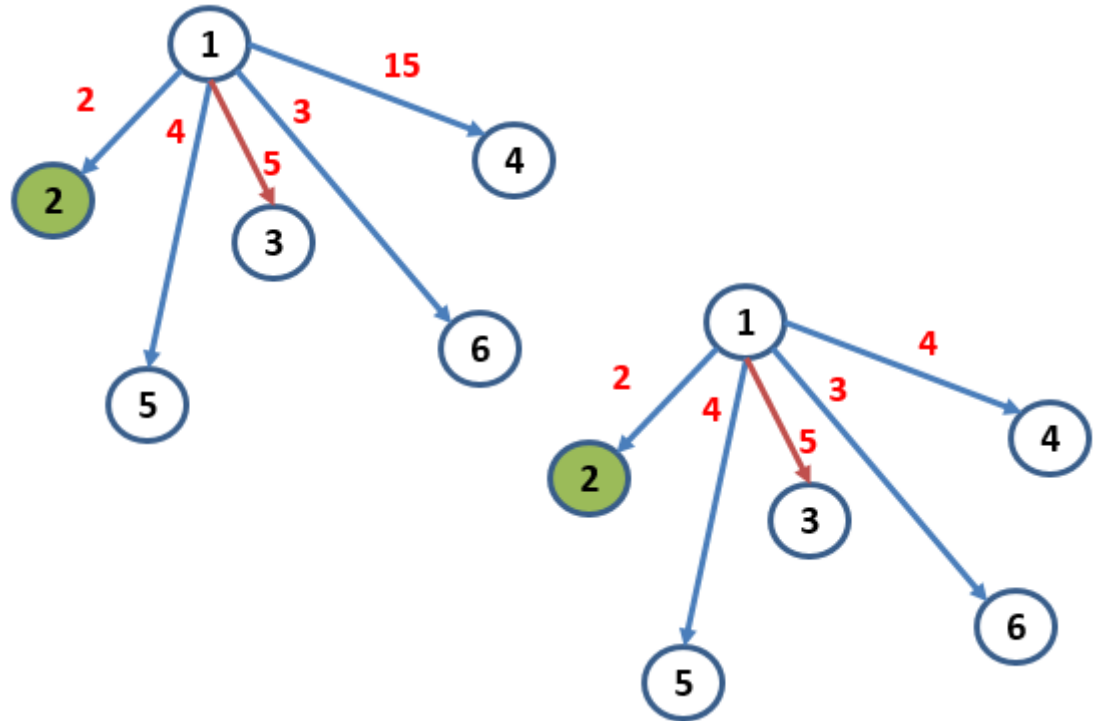
Camino desde "1" a "5" directo es 1, pasando por "2" el costo es "4" también por lo tanto no mejora
Camino desde "1" a "3" directo no existe, pasando por "2" el costo es "5" el cual es mejor a no existir
Camino desde "1" a "6" directo es 3, pasando por "2" no existe, queda el que está
Camino desde "1" a "4" directo es 15, pasando por "2" no existe, queda el que está

Greedy Grafos – Dijkstra - Ejemplo

Grafo Original



Grafo Resultado



Se selecciona el más cercano que es el nodo "6" por tener el peso menor "3" entre los que quedan

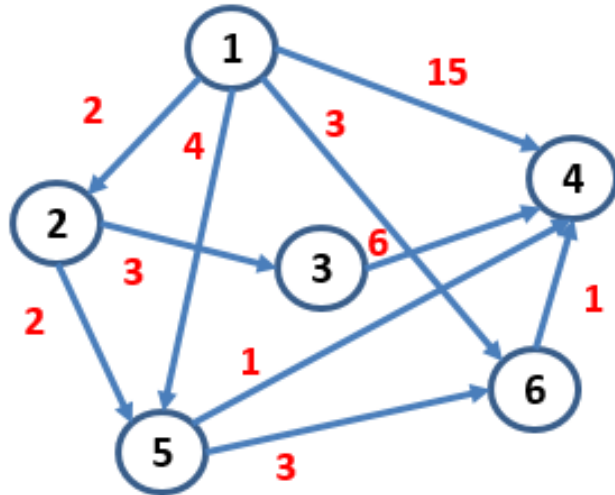
Camino desde "1" a "5" directo es 4, pasando por "6" no existe, queda el que está

Camino desde "1" a "3" directo es 5, pasando por "6" no existe, queda el que está

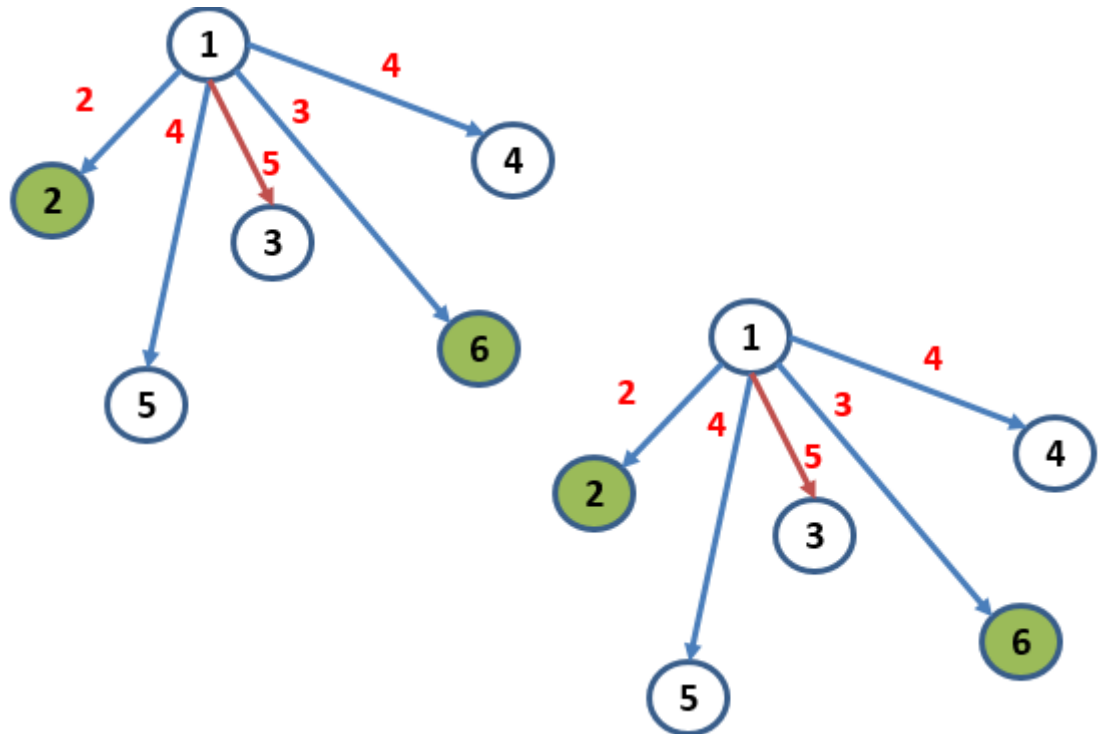
Camino desde "1" a "4" directo es 15, pasando por "6" el costo es "4", por lo cual mejora y se actualiza

Greedy Grafos – Dijkstra - Ejemplo

Grafo Original



Grafo Resultado



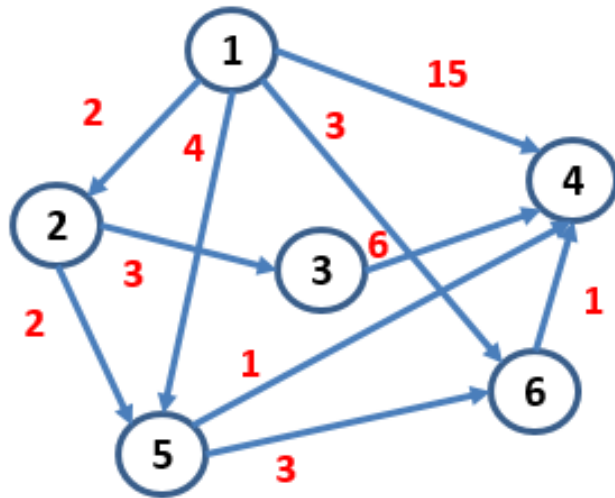
Se selecciona el más cercano que es el nodo "5" ó "4" por tener el peso menor "4", es indistinto cual, vamos a seleccionar 5.

Camino desde "1" a "3" directo es 5, pasando por "5" no existe, queda el que está

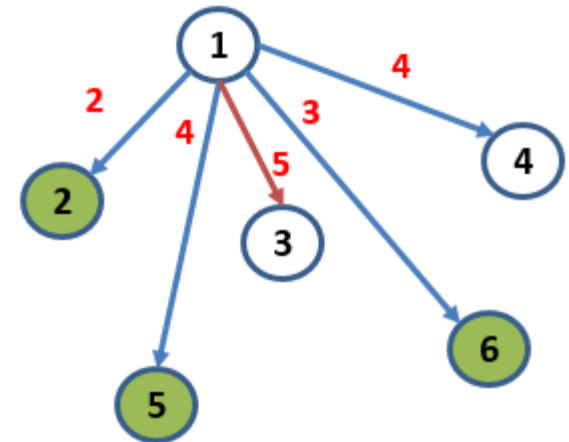
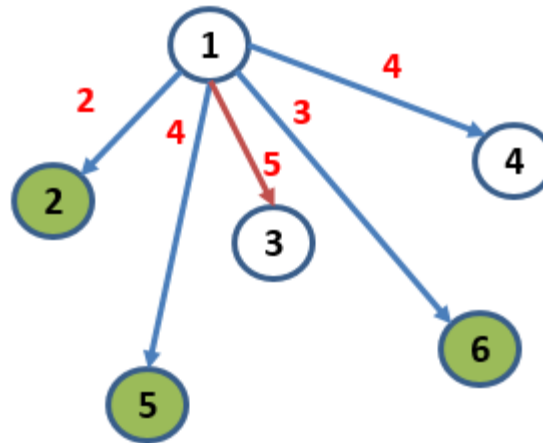
Camino desde "1" a "4" directo es 4, pasando por "5" el costo es "5", por lo cual no mejora, se deja el que esta

Greedy Grafos – Dijkstra - Ejemplo

Grafo Original



Grafo Resultado

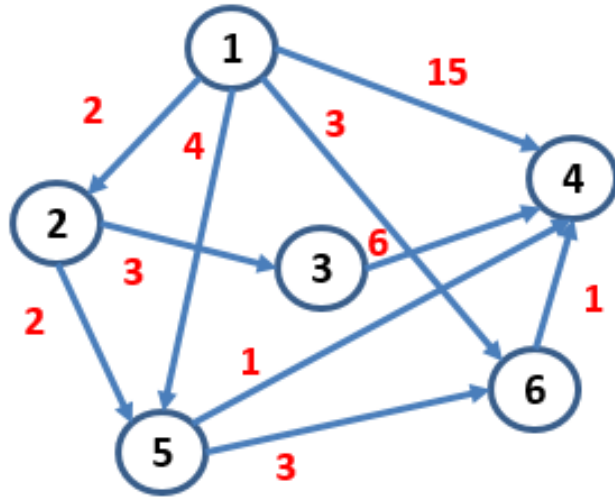


Se selecciona el más cercano que es el nodo "4" por tener el peso menor "4"

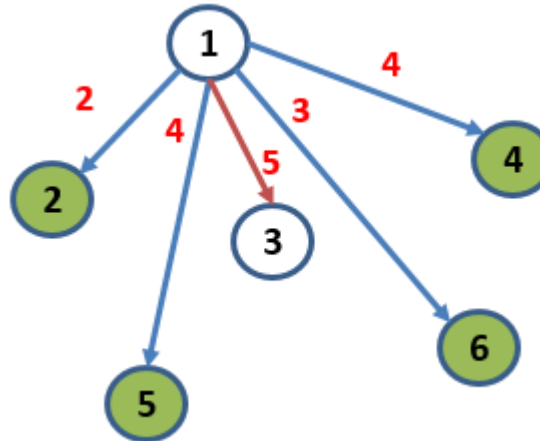
Camino desde "1" a "3" directo es 5, pasando por "4" no existe, queda el que está

Greedy Grafos – Dijkstra - Ejemplo

Grafo Original



Grafo Resultado

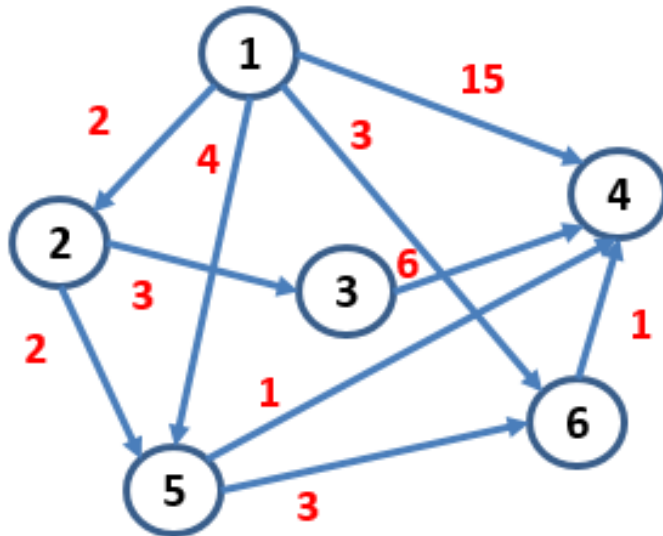


Se selecciona el más cercano que es el nodo "3" por tener el peso menor "5"

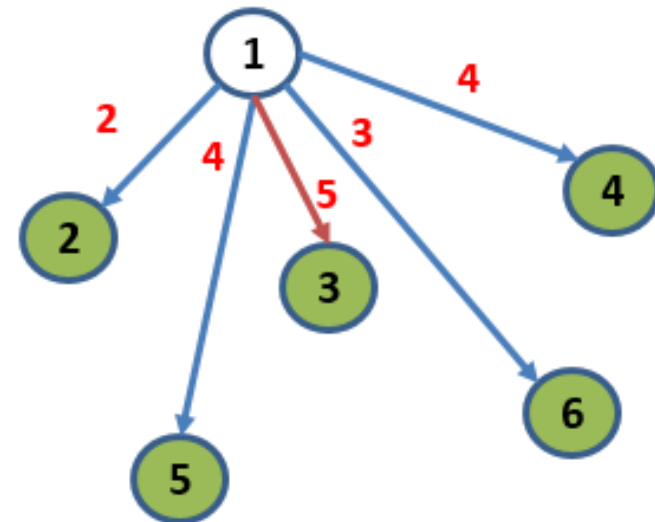
No quedan nodos por los cuales pasar, por lo tanto, se obtuvo el resultado final

Greedy Grafos – Dijkstra - Ejemplo

Grafo Original



Grafo Resultado FINAL



Greedy Grafos – Dijkstra - Algoritmo

ALGORITMO DIJKSTRA

Entrada: G : Grafo<entero>, v : entero

Salida: A Grafo<entero>

//Paso 1 //Conjunto de Vértices ya visitados

Conjunto<entero> $Visitados \leftarrow \{v\}$

//Paso 2 //Grafo auxiliar

Grafo<entero> $A \leftarrow inicializarGrafo()$;

para cada $w \in Vertices(G)$

$agregarVertice(A, w)$

fin para

para cada $v' \in Adyacentes(G, v)$

$agregarArista(A, v, v', Peso(G, v, v'))$

fin para

//Conjunto de Vértices pendientes de calculo

$Pendientes \leftarrow Vertices(G) \setminus Visitados$

Greedy Grafos – Dijkstra - Algoritmo

```
//Paso 3
mientras Pendientes  $\neq \emptyset$ 
     $w \leftarrow n : \text{PesoArista}(A, v, n) = \min\{\text{PesoArista}(A, v, p) \mid \forall n, p \in \text{Pendientes}\}$ 
    Visitados  $\leftarrow \text{Visitados} \cup \{w\}$ 
    Pendientes  $\leftarrow \text{Pendientes} \setminus \{w\}$ 
    auxPendientes  $\leftarrow \text{Pendientes}$ 
    mientras auxPendientes  $\neq \emptyset$ 
         $p \leftarrow \text{elegir}(\text{auxPendientes})$ 
        auxPendientes  $\leftarrow \text{auxPendientes} \setminus \{p\}$ 
        si existeArista(A, v, w) Y existeArista(G, w, p)
            si existeArista(A, v, p)
                si  $\text{pesoArista}(A, v, w) + \text{pesoArista}(G, w, p) < \text{pesoArista}(A, v, p)$ 
                    agregarArista(A, v, p,  $\text{pesoArista}(A, v, w) + \text{pesoArista}(G, w, p)$ )
                fin si
            sino
                agregarArista(A, v, p,  $\text{pesoArista}(A, v, w) + \text{pesoArista}(G, w, p)$ )
            fin si
        fin si
    fin mientras
fin mientras
devolver A
```

Greedy Grafos – Dijkstra – Complejidad Temporal

ALGORITMO DIJKSTRA

Entrada: G : Grafo<entero>, v : entero

Salida: A Grafo<entero>

//Paso 1 //Conjunto de Vértices ya visitados

Conjunto<entero> $Visitados \leftarrow \{v\}$

//Paso 2 //Grafo auxiliar

Grafo<entero> $A \leftarrow inicializarGrafo()$;

para cada $w \in Vertices(G)$

 agregarVertice(A, w)

fin para

para cada $v' \in Adyacentes(G, v)$

 agregarArista($A, v, v', Peso(G, v, v')$)

fin para

//Conjunto de Vértices pendientes de calculo

$Pendientes \leftarrow Vertices(G) \setminus Visitados$

Cada ciclo depende de la cantidad de nodos o vértices, y los adyacentes, que en el peor de los casos itera en nodos $- 1$, por lo tanto la complejidad temporal es:

$$\mathcal{O}(n)$$

Nota: se consideran las complejidades de las operaciones sobre grafos constante, con una matriz de acceso directo.

Greedy Grafos – Dijkstra – Complejidad Temporal

//Paso 3

mientras $Pendientes \neq \emptyset$

$w \leftarrow n : \text{PesoArista}(A, v, n) = \min\{\text{PesoArista}(A, v, p)\} \forall n, p \in$

$Pendientes$

$Visitados \leftarrow Visitados \cup \{w\}$

$Pendientes \leftarrow Pendientes \setminus \{w\}$

$auxPendientes \leftarrow Pendientes$

mientras $auxPendientes \neq \emptyset$

$p \leftarrow \text{elegir}(auxPendientes)$

$auxPendientes \leftarrow auxPendientes \setminus \{p\}$

si $\text{existeArista}(A, v, w)$ Y $\text{existeArista}(G, w, p)$

si $\text{existeArista}(A, v, p)$

si $\text{pesoArista}(A, v, w) + \text{pesoArista}(G, w, p) <$

$\text{pesoArista}(A, v, p)$

$\text{agregarArista}(A, v, p, \text{pesoArista}(A, v, w) +$

$\text{pesoArista}(G, w, p)$

fin si

sino

$\text{agregarArista}(A, v, p, \text{pesoArista}(A, v, w) + \text{pesoArista}(G, w, p)$

fin si

fin si

fin mientras

fin mientras

devolver A

Dos ciclos anidados que dependen de la cantidad de nodos, por lo tanto iteran n veces, siendo n la cantidad de vértices. Dado lo anterior la complejidad temporal es:

$$O(n^2)$$



Qué es un árbol de recubrimiento

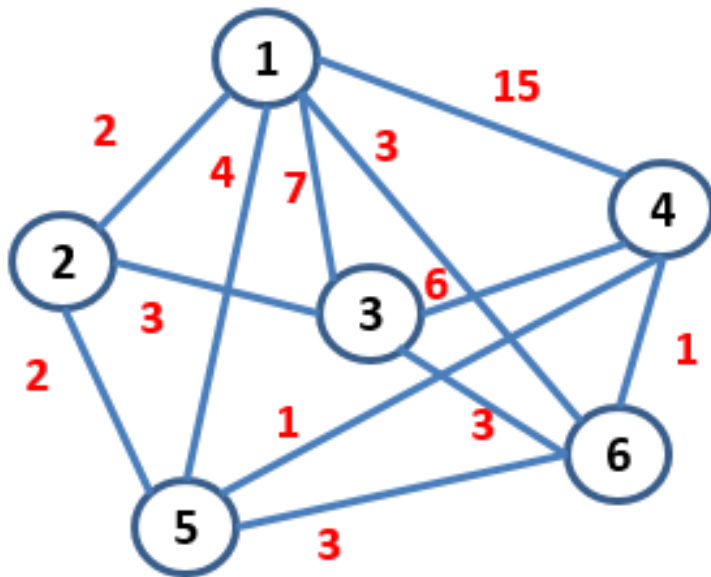


Árbol de recubrimiento de costo mínimo

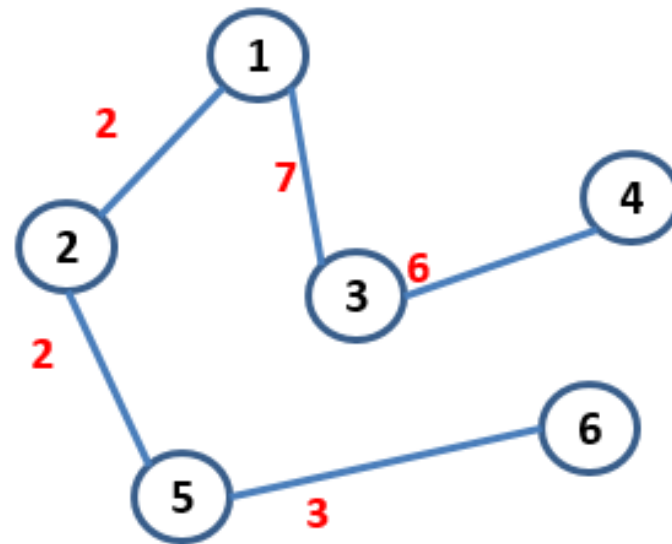
- Los árboles de recubrimiento se trabajan con grafos no dirigidos.
- Un árbol de recubrimiento de un grafo $G(V;A)$ a un grafo $G'(V;A')$ que contiene los mismos vértices que G y un subconjunto de las aristas, que forma un árbol, es decir, G' es un grafo conexo y acíclico.
- Árbol de recubrimiento mínimo es al árbol de recubrimiento que tenga el menor costo de todos, es decir que la suma de sus aristas sea la mínima.
- Para algunos grafos, puede existir más de un árbol de recubrimiento mínimo, todos con el mismo costo.

Árbol de recubrimiento Mínimo – Ejemplo 1

Grafo original



Ej. árbol recubrimiento

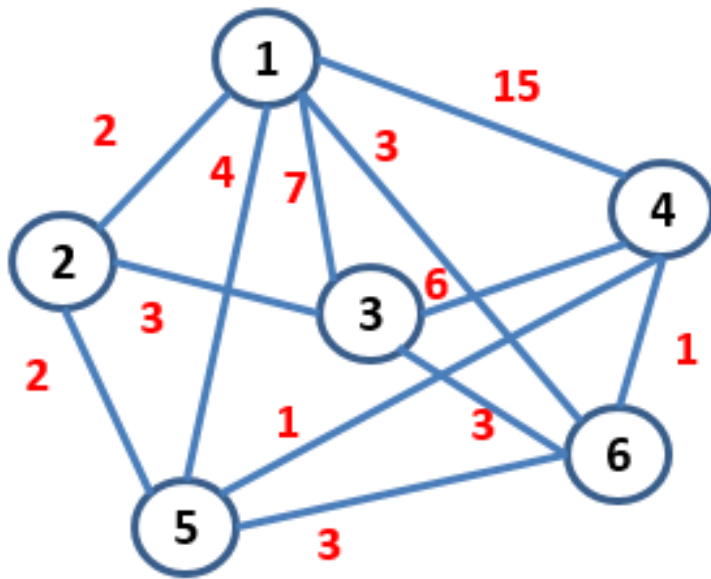


Costo: 20

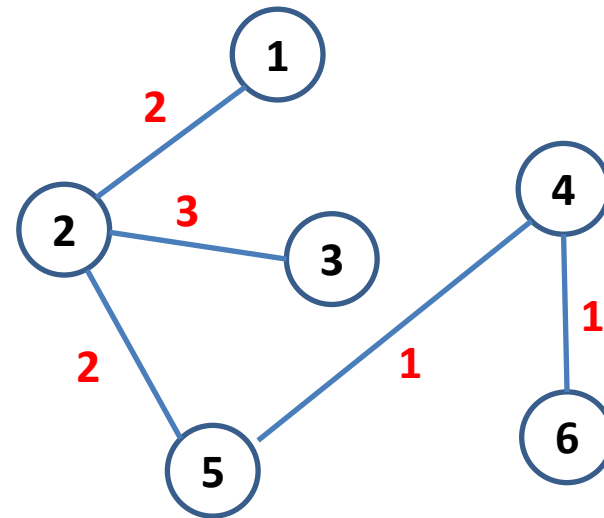
No es el árbol de recubrimiento de costo mínimo

Árbol de Recubrimiento Mínimo – Ejemplo 2

Grafo original



Ej. árbol recubrimiento



Costo: 9

Es el árbol de recubrimiento de costo mínimo

Árbol de Recubrimiento Mínimo - Algoritmos

- Existen dos algoritmos con estrategia ***Greedy*** que permite obtener un árbol de cubrimiento de un grafo con costo mínimo.
- Los algoritmos son:
 - **PRIM**
 - **Kruskal**
- En el caso de que se tenga un único árbol de recubrimiento de costo mínimo, ambos algoritmos arrojan el mismo resultado. Si no, podría variar por la estrategia que sigue cada uno.



Qué es el algoritmo de PRIM



Algoritmo de PRIM

La estrategia del algoritmo de PRIM para hallar el árbol de recubrimiento mínimo consiste en ir armando un árbol a partir de un vértice cualquiera del grafo, e ir agregando vértices hasta completar todos sus nodos.

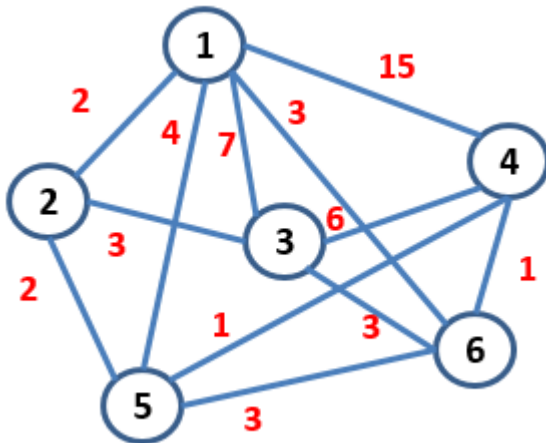
Estos vértices son seleccionados por el criterio que tenga un menor costo con otro nodo previamente seleccionado.

Algoritmo de PRIM - Estrategia

- **Candidatos:** los nodos del grafo.
- **Función de selección:** la estrategia para ir agregando vértices es ir agregando el nodo que, no estando agregado aún, sea el que tenga la arista de costo mínimo con algún otro vértice ya seleccionado.
- **La función factibilidad:** dependiendo la implementación es decidir si se utiliza la arista obtenida en la selección o no.
- **La función solución:** haber trabajado con todos los nodos candidatos.
- **La función objetivo:** árbol de recubrimiento de costo mínimo.

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



Tratados

{}

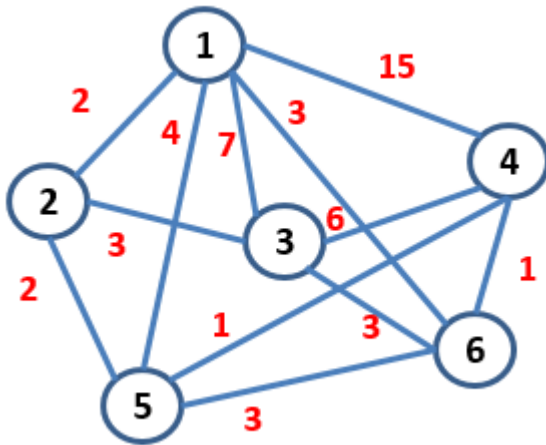
Pendientes

{1,2,3,4,5,6}

Selecciono = 2

Algoritmo de PRIM - Ejemplo

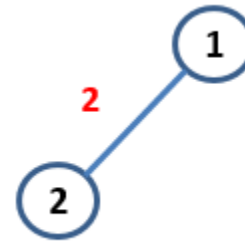
Grafo original



Tratados

$\{2\}$

Árbol recubrimiento mínimo



Pendientes

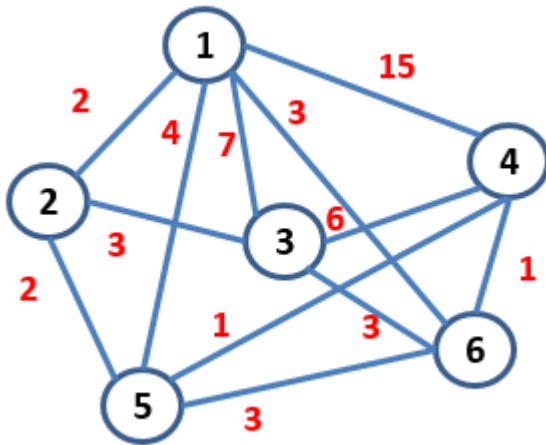
$\{1,3,4,5,6\}$

Selecciono = 1 porque es el de menor costo entre tratados y pendientes

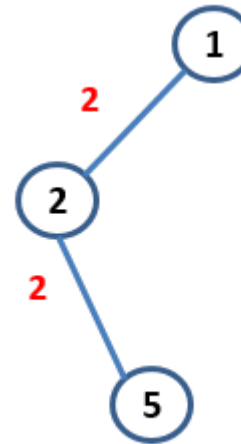
También se puede seleccionar el 5 porque tiene el mismo costo de arista

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



Tratados

$\{2,1\}$

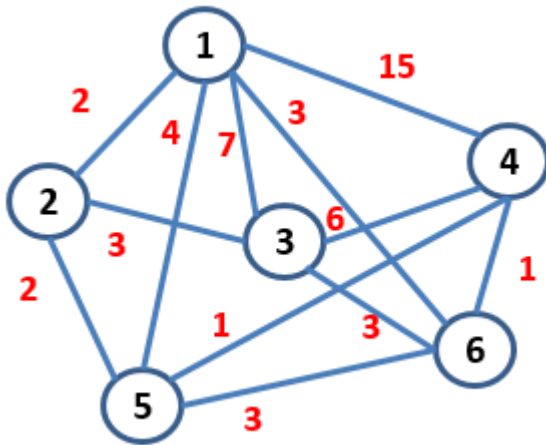
Pendientes

$\{3,4,5,6\}$

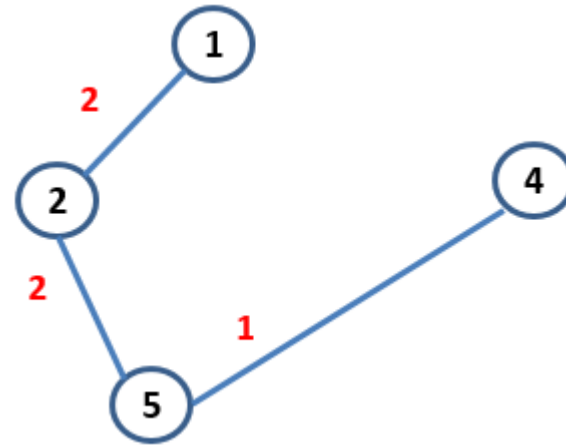
Selecciono = 5 porque es el de menor costo entre tratados y pendientes

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



Tratados

$\{2,1,5\}$

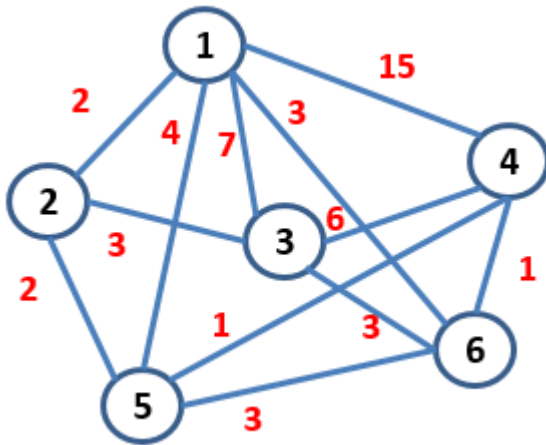
Pendientes

$\{3,4,6\}$

Selecciono = 4 porque es el de menor costo entre tratados y pendientes

Algoritmo de PRIM - Ejemplo

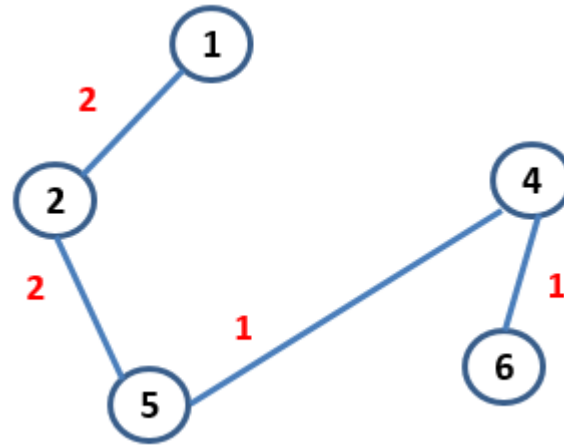
Grafo original



Tratados

$\{2,1,5,4\}$

Árbol recubrimiento mínimo



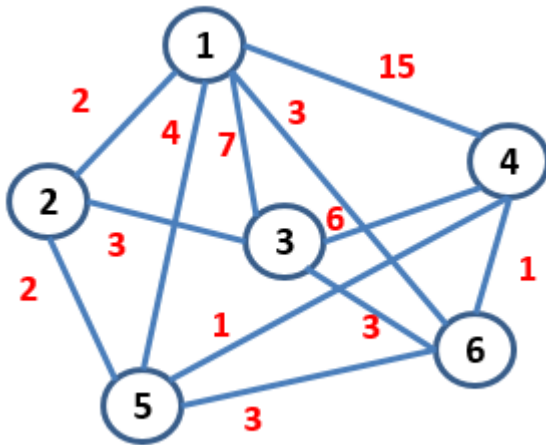
Pendientes

$\{3,6\}$

Selecciono = 6 porque es el de menor costo entre tratados y pendientes

Algoritmo de PRIM - Ejemplo

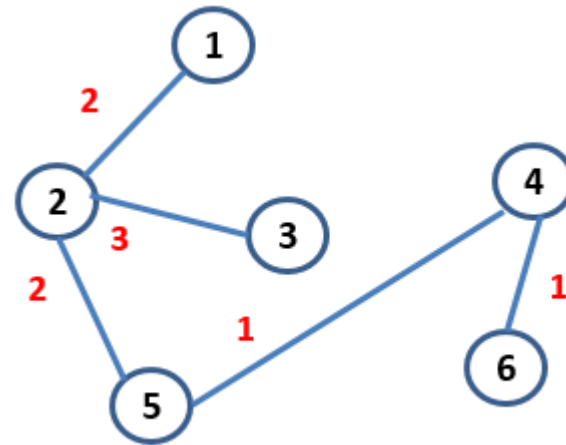
Grafo original



Tratados

$\{2,1,5,4,6\}$

Árbol recubrimiento mínimo



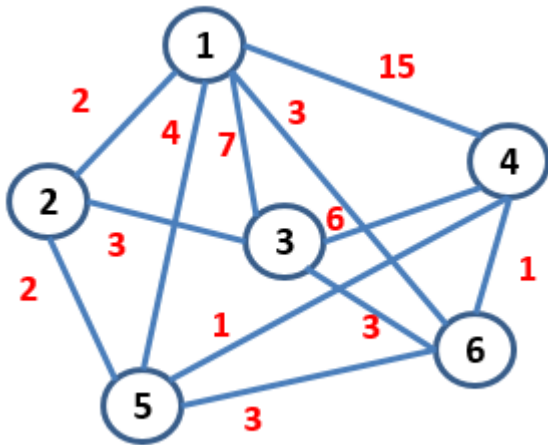
Pendientes

$\{3\}$

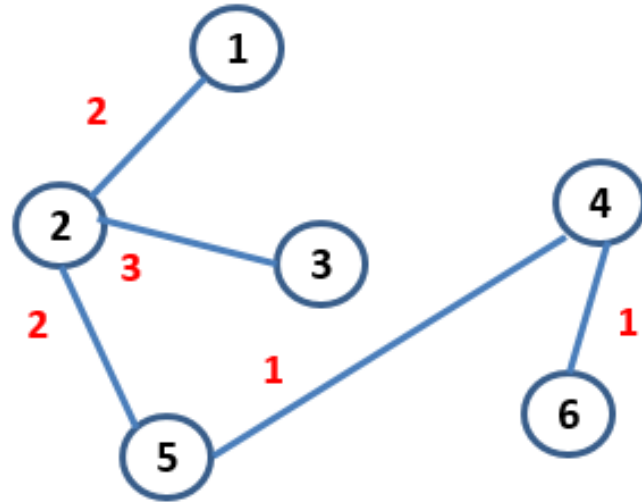
Selecciono = 3 porque es el de menor costo entre tratados y pendientes
en éste caso hay dos opciones porque hay dos aristas con costo 3.
Lo anterior implica que hay más de un árbol de recubrimiento de
costo mínimo

Algoritmo de PRIM - Ejemplo

Grafo original



Árbol recubrimiento mínimo



Costo Árbol Recubrimiento Mínimo= 9

Algoritmo de PRIM - Algoritmo

ALGORITMO PRIM

Entrada: G : grafo de origen

alida: R : grafo solución

inicializar(R)

para cada $v \in Vertices(G)$

agregarVertice(R, v)

fin para

$vertice \leftarrow elegir(Vertices(R))$

$pendientes \leftarrow Vertices(R) \setminus \{vertice\}$

para cada $w \in Adyacentes(G, vertice)$

agregarArista($R, vertice, w, pesoArista(G, vertice, w)$)

fin para

mientras $pendientes \neq \emptyset$

$vertice = v : pesoArista(v, v_i) = \min(pesoArista(v_1, v_2) : \forall v_1 \in$

$pendientes, v_2 \notin pendientes$

$sacar(pendientes, vertice)$

para cada $p \in pendientes$

si $existeArista(G, p, vertice)$ Y NO $existeArista(R, p, x) \forall x$

agregarArista($R, p, vertice, pesoArista(G, p, vertice)$)

sino

si $existeArista(G, p, vertice)$ Y $pesoArista(R, p, x) >$

$pesoArista(G, p, vertice)$

$eliminarArista(R, p, x)$

agregarArista($R, p, vertice, pesoArista(G, p, vertice)$)

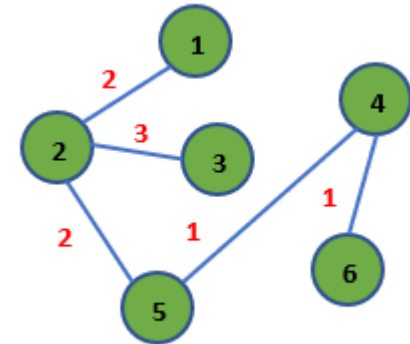
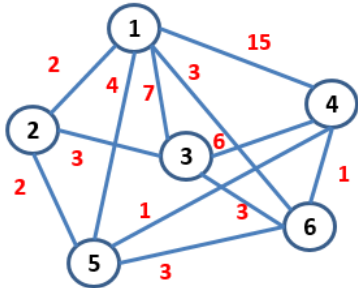
fin si

fin si

fin para

fin mientras

devolver R



Algoritmo de PRIM – Complejidad Temporal

ALGORITMO PRIM

Entrada: G : grafo de origen

Salida: R : grafo solución

inicializar(R)

para cada $v \in \text{Vertices}(G)$

agregarVertice(R, v)

fin para

$\text{vertice} \leftarrow \text{elegir}(\text{Vertices}(R))$

$\text{pendientes} \leftarrow \text{Vertices}(R) \setminus \{\text{vertice}\}$

para cada $w \in \text{Adyacentes}(G, \text{vertice})$

agregarArista($R, \text{vertice}, w, \text{pesoArista}(G, \text{vertice}, w)$)

fin para

mientras $\text{pendientes} \neq \emptyset$

$\text{vertice} = v : \text{pesoArista}(v, v_i) = \min(\text{pesoArista}(v_1, v_2) : \forall v_1 \in \text{pendientes}, v_2 \notin \text{pendientes})$

sacar($\text{pendientes}, \text{vertice}$)

para cada $p \in \text{pendientes}$

si $\text{existeArista}(G, p, \text{vertice})$ Y NO $\text{existeArista}(R, p, x) \forall x$

agregarArista($R, p, \text{vertice}, \text{pesoArista}(G, p, \text{vertice})$)

sino

si $\text{existeArista}(G, p, \text{vertice})$ Y $\text{pesoArista}(R, p, x) >$

$\text{pesoArista}(G, p, \text{vertice})$

eliminarArista(R, p, x)

agregarArista($R, p, \text{vertice}, \text{pesoArista}(G, p, \text{vertice})$)

fin si

fin si

fin para

fin mientras

devolver R

Los ciclos dependen de la cantidad de vértices, teniendo un ciclo anidado.

$$O(n^2)$$



Qué es el algoritmo de Kruskal



Algoritmo de Kruskal

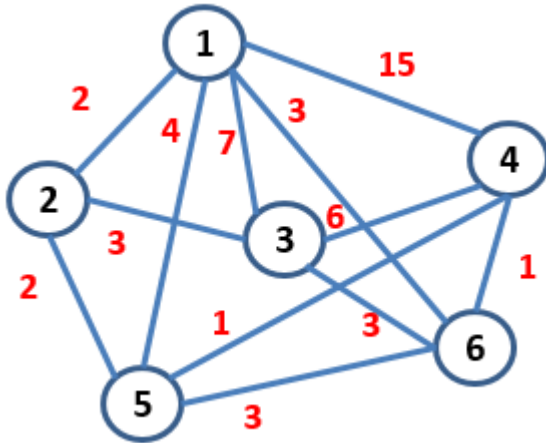
- El algoritmo de Kruskal se basa en ir armando árboles pequeños e ir uniéndolos hasta obtener un único árbol de recubrimiento cuyo costo sea mínimo.
- Se comienza armando un árbol con un único nodo por cada vértice. Luego se toman las aristas del grafo original, ordenadas por su peso, de menor a mayor. Para cada arista, si tiene cada extremo en un árbol diferente, se agrega dicha arista al resultado y se unen los dos árboles que contienen sus extremos.

Algoritmo de Kruskal - Estrategia

- **Candidatos:** todas las aristas del grafo.
- **Función de selección:** seleccionar la arista de menor valor.
- **La función factibilidad:** la arista no debe tener los nodos en un mismo subárbol.
- **La función solución:** haber trabajado con todas las aristas.
- **La función objetivo:** árbol de recubrimiento de costo mínimo.

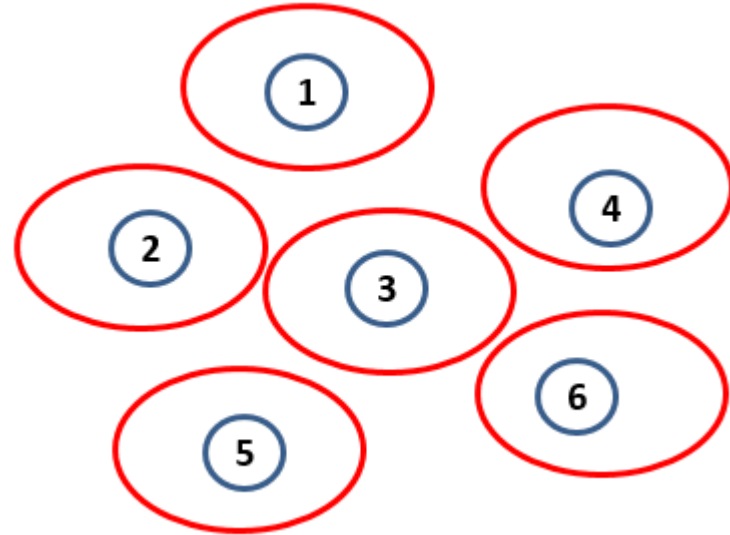
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

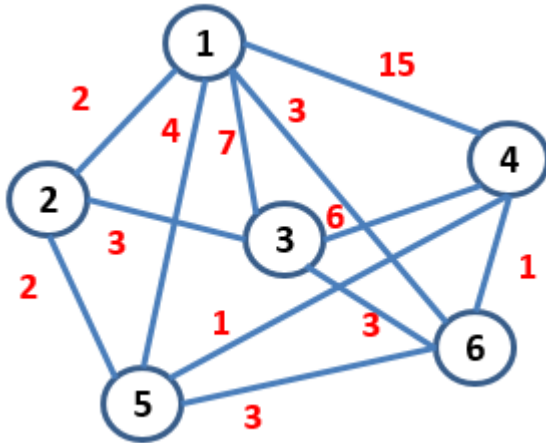
Árbol recubrimiento mínimo



Inicialmente cada nodo está en un subárbol y las aristas ordenadas por costo de menor a mayor.

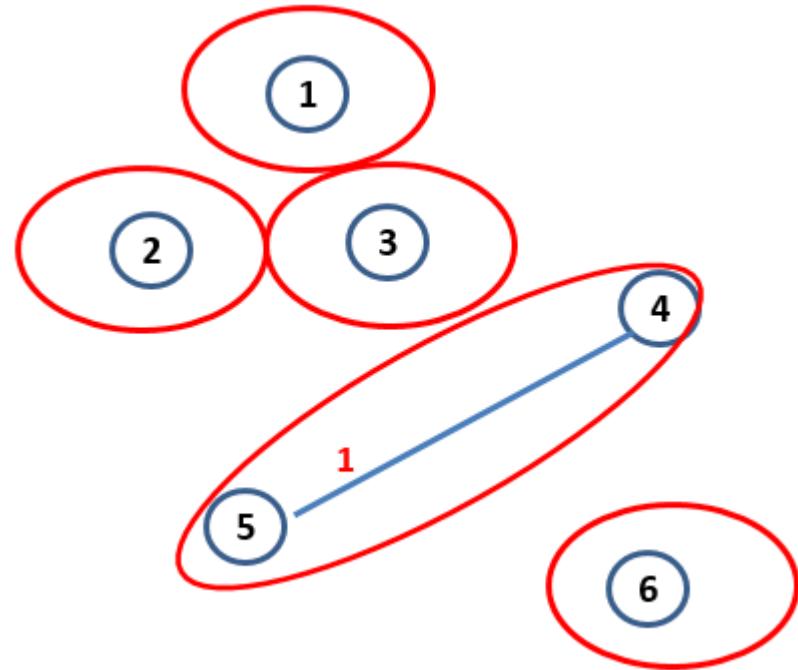
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

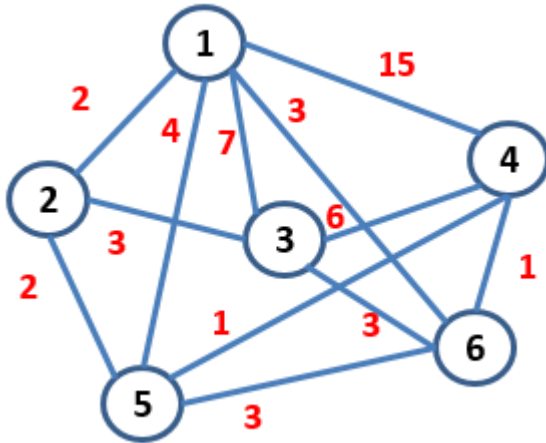
Árbol recubrimiento mínimo



Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

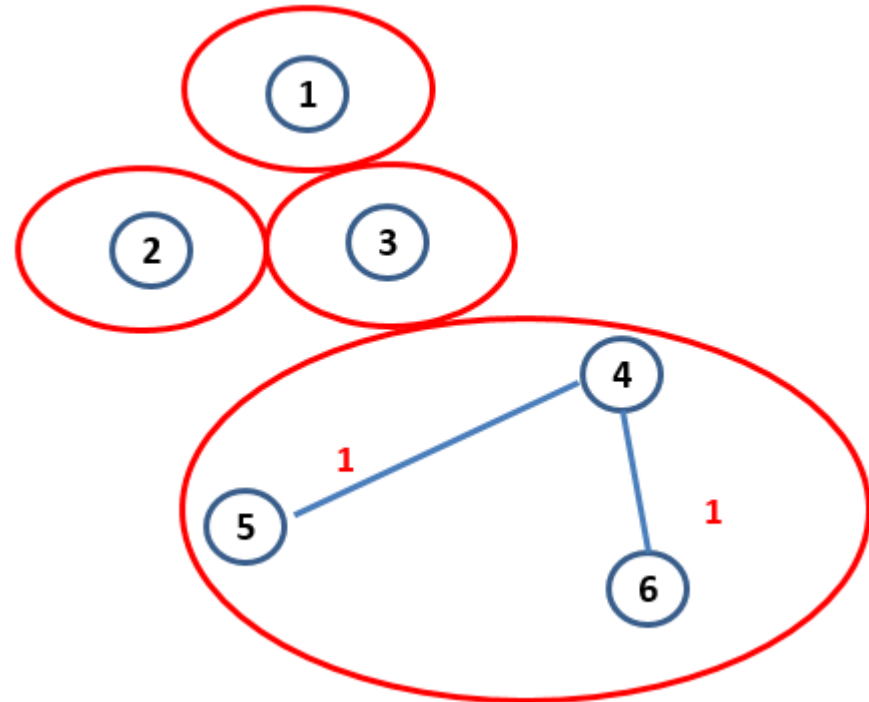
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

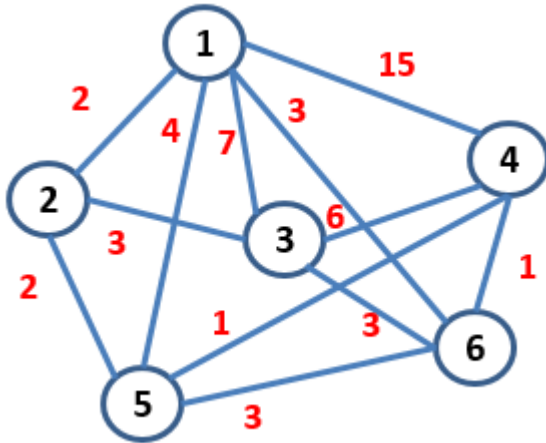
Árbol recubrimiento mínimo



Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

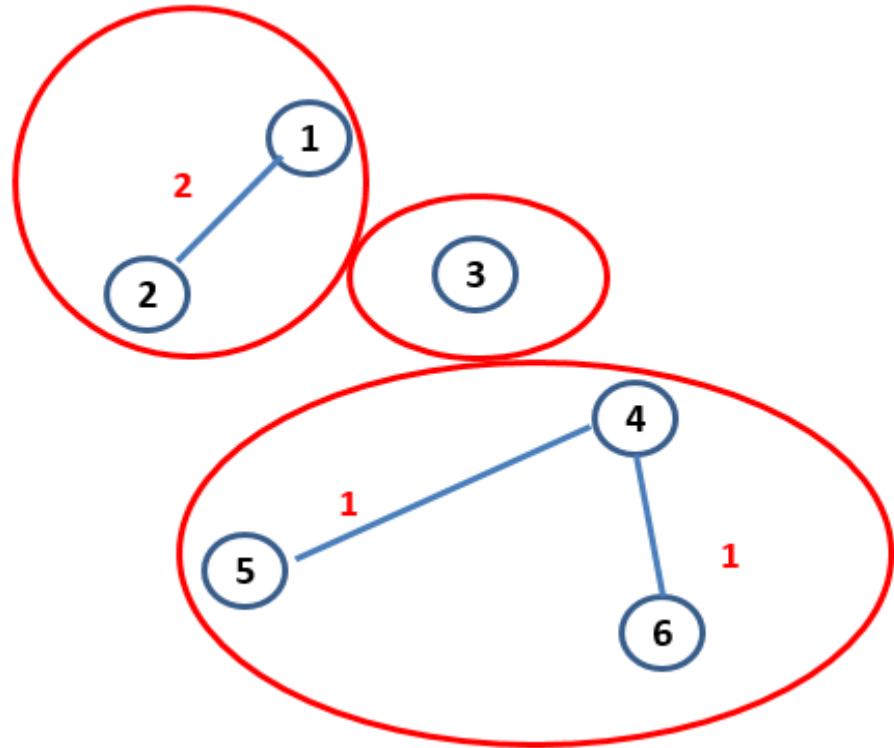
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

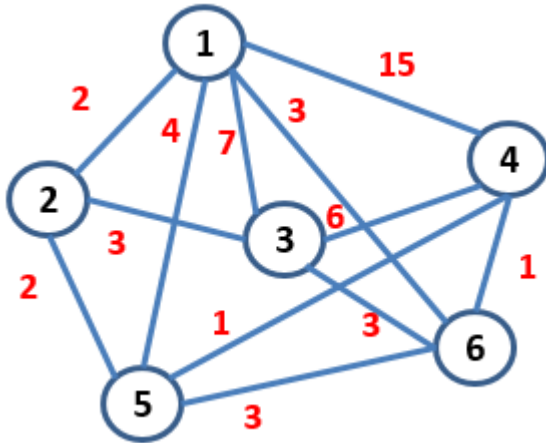
Árbol recubrimiento mínimo



Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

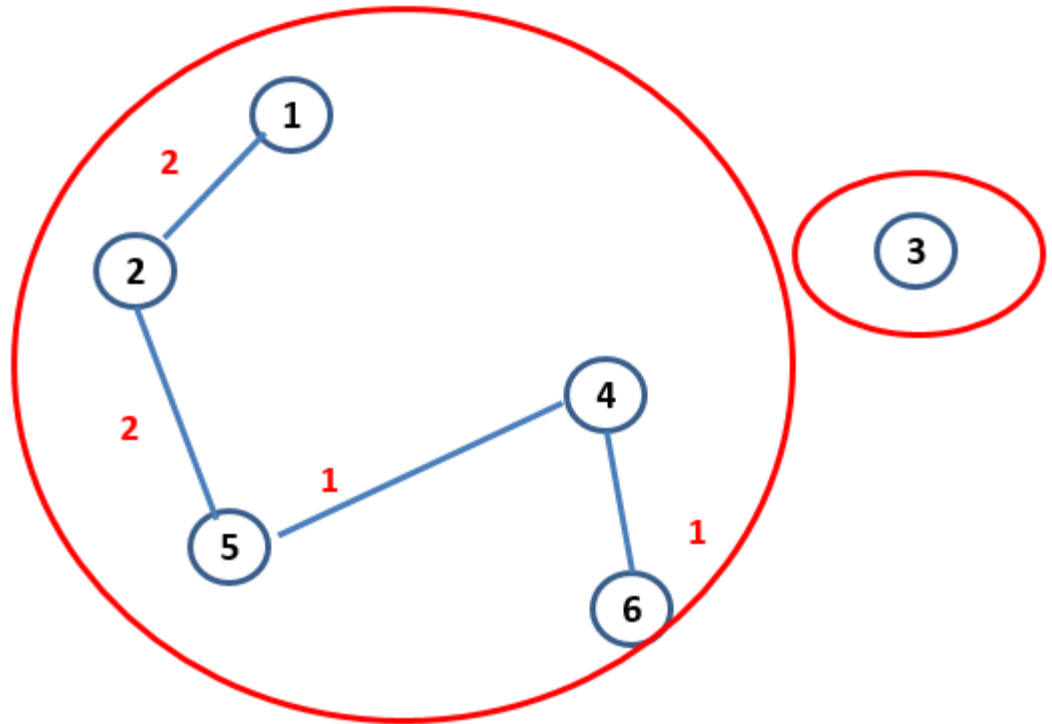
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

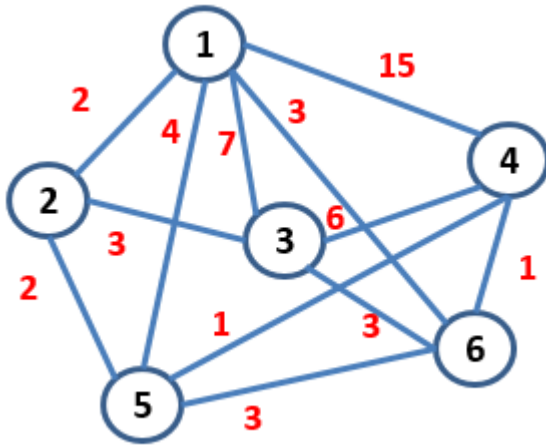
Árbol recubrimiento mínimo



Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

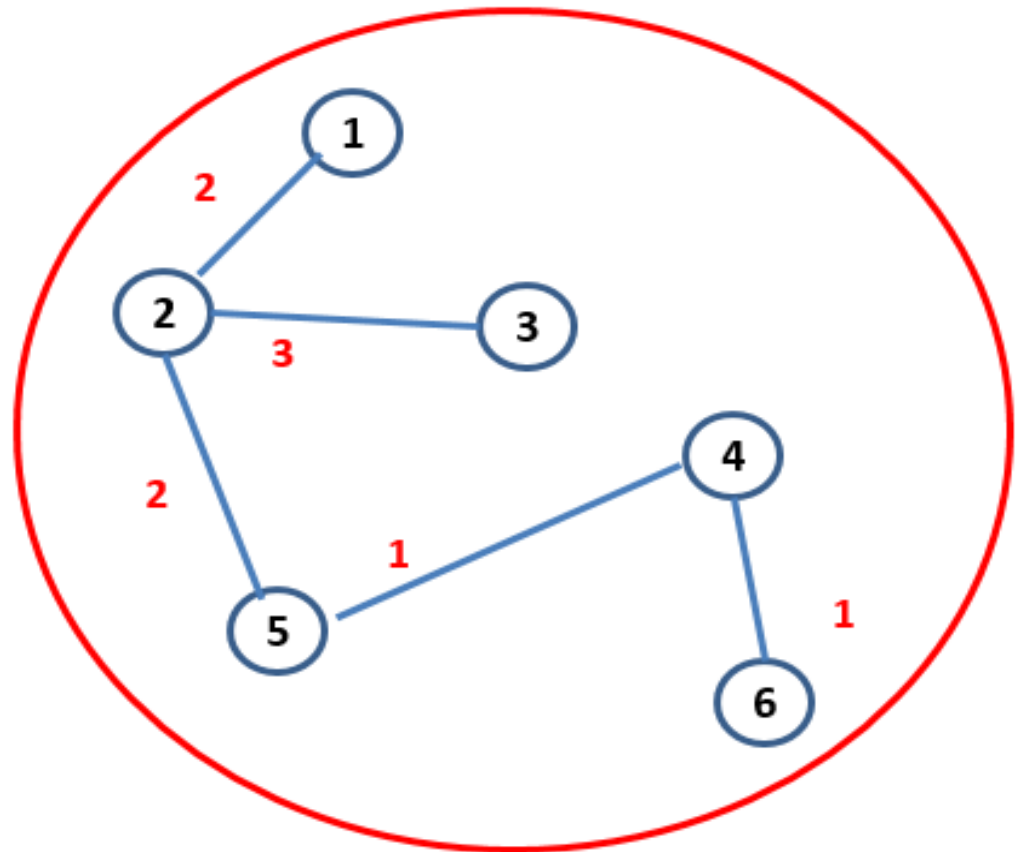
Algoritmo de Kruskal - Ejemplo

Grafo original



Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Árbol recubrimiento mínimo



Se selecciona la arista de menor tamaño entre las que quedan y, si es factible, se utiliza.

Algoritmo de Kruskal - Algoritmo

ALGORITMO KRUSKAL

Entrada: G : grafo de origen

Salida: R : grafo solución

$inicializar(R)$

$E \leftarrow inicializarConjuntos()$

$cantidadVertices \leftarrow 0$

para cada $v \in Vertices(G)$

$agregarVertice(R, v)$

$E[v] = v$

$cantidadVertices \leftarrow cantidadVertices + 1$

fin para

$C \leftarrow inicializarColaPrioridad()$

para cada $v1 \in Vertices(G)$

 para cada $v2 \in Vertices(G)$

 si $existeArista(G, v1, v2)$ Y NO $existe(C, Arista(v1, v2))$

$agregar(C, Arista(v1, v2), pesoArista(G, v1, v2))$

 fin si

 fin para

fin para

Indice (vertice)	Nro. Componente
1	1
2	2
3	3
4	4
5	5
6	6

Continúa en próxima slide

Algoritmo de Kruskal – Algoritmo (Cont.)

mientras $\text{cantidadVertices} > 1$

$a \leftarrow \text{menor}(C)$

 eliminarMenor(C)

 si $E[a.\text{origen}] \neq E[a.\text{destino}]$

$\text{cantidadVertices} \leftarrow \text{cantidadVertices} - 1$ //unifico los conjuntos

 para cada $v \in E$

 si $E[v] = E[a.\text{destino}]$

$E[v] \leftarrow E[a.\text{origen}]$

 fin si

 fin para

 agregarArista($R, a.\text{origen}, a.\text{destino}, a.\text{peso}$)

 fin si

fin mientras

devolver R

Indice (vertice)	Nro. Componente
1	2
2	2
3	2
4	2
5	2
6	2

Origen	Destino	Costo
5	4	1
6	4	1
2	1	2
2	5	2
1	6	3
2	3	3
3	6	3
5	6	3
1	5	4
3	4	6
1	3	7
1	4	15

Algoritmo de Kruskal – Complejidad Temporal

ALGORITMO KRUSKAL

Entrada: G : grafo de origen

Salida: R : grafo solución

inicializar(R)

$E \leftarrow \text{inicializarConjuntos}()$

cantidadVertices $\leftarrow 0$

para cada $v \in \text{Vertices}(G)$

agregarVertice(R, v)

$E[v] = v$

cantidadVertices $\leftarrow \text{cantidadVertices} + 1$

fin para

$C \leftarrow \text{inicializarColaPrioridad}()$

para cada $v1 \in \text{Vertices}(G)$

 para cada $v2 \in \text{Vertices}(G)$

 si *existeArista*($G, v1, v2$) Y NO *existe*($C, \text{Arista}(v1, v2)$)

agregar($C, \text{Arista}(v1, v2), \text{pesoArista}(G, v1, v2)$)

 fin si

 fin para

fin para

Aquí itera por la cantidad de vértices.

Aquí itera por la cantidad de vértices, más el costo de agregar una arista a la cola de prioridad.

Continúa en próxima *slide*

Algoritmo de Kruskal – Complejidad Temporal (Cont.)

```
mientras cantidadVertices > 1
  a ← menor(C)
  eliminarMenor(C)
  si E[a.origen] <> E[a.destino]
    cantidadVertices ← cantidadVertices – 1 //unifico los conjuntos
    para cada v ∈ E
      si E[v] = E[a.destino]
        E[v] = E[a.origen]
      fin si
    agregarArista(R, a.origen, a.destino, a.peso)
  fin si
fin mientras
devolver R
```

Aquí itera por la cantidad de vértices.

$\Theta(n^2 \log a)$ donde “n” es la cantidad de vértices y “a” la cantidad de aristas, si la cantidad de aristas tiende a n^2 entonces se puede decir que la complejidad es:

$\Theta(a \log a)$



¡Muchas gracias!

